

计网复习

- 计网复习
 - 概念
 - 协议 (protocol)
 - 端系统
 - 接入网
 - 边缘路由器
 - 物理媒体
 - 分组交换
 - 存储转发传输
 - 排队时延和分组丢失
 - 转发表
 - 时延
 - 应用程序体系结构
 - 进程通信
 - 套接字
 - 进程寻址
 - 对传输服务的要求
 - 传输层协议
 - TLS
 - HTTP
 - 报文格式
 - Cookie
 - SMTP
 - MIME
 - POP3 & IMAP
 - DNS
 - P2P
 - 视频流
 - 编码
 - DASH
 - 客户端缓冲和播放延迟
 - CDN
 - OTT
 - 传输层
 - 端口号
 - TCP
 - UDP
 - 可靠数据传输
 - TCP 协议
 - MSS
 - 流量控制
 - 三次握手
 - 释放连接
 - 拥塞控制
 - AIMD
 - TCP拥塞控制算法
 - TCP Cubic
 - 网络层
 - 网络服务模型
 - 路由器工作原理
 - 基于目的地转发
 - 排队

- 分组调度
- IPv4
 - 首部检验和
 - IP分片fragment
 - IP地址
 - 划分子网
 - 获取IP地址 DHCP
 - NAT
- IPv6
- 隧道
- 冒号十六进制表示法
- 通用转发
- OpenFlow
- 路由协议
 - 链路状态路由选择算法
 - 距离向量路由选择算法
 - 两者比较
- 可扩展路由
 - OSPF
- BGP
- 热土豆路由
- SDN
- ICMP
- 网络管理
 - SNMP
- 链路层
- 差错检测
 - 奇偶校验
 - Internet校验和
 - CRC
- 多路接入链路&协议
 - 信道划分
 - 随机接入
 - ALOHA(Slotted ALOHA) 时隙ALOHA
 - 纯ALOHA
 - CSMA
 - 轮流
- MAC地址
- ARP
- 以太网
- 以太网帧结构
- 交换机
 - 自学习
 - 对比路由器
- VLAN
- 无线链路
- 802.11 WLAN
- 碰撞避免: RTS/CTS
- 802.11帧结构
- 802.11 同一子网内移动性
- 802.11 高级
- 计算
 - 传输时间
 - 缓存
 - P2P计算
 - pipelining
 - 慢开始和拥塞避免

- [路由器缓存](#)
- [时隙ALOHA效率](#)
- [术语缩写对照表](#)

概念

协议 (protocol)

定义了在两个或多个通信实体之间交换的报文的格式和顺序，以及报文发送和 / 或接收一条报文或其他事件所采取的动作

端系统

通常把与因特网相连的计算机和其他设备称为端系统。

接入网

将端系统物理连接到其边缘路由器的网络

边缘路由器

端系统到任何其他远程端系统的路径上的第一台路由器

物理媒体

双绞铜线、同轴电缆、多模光纤缆、陆地无线电频谱和卫星无线电频谱

分组交换

主机将应用层消息分解为数据包/分组

将报文分成较小的数据块，称为**分组**，并为每个分组添加首部字节。每个分组独立地从源端系统到目的端系统通过通信链路和分组交换机传输。

交换机主要有两类：**路由器** 和 **链路层交换机**

存储转发传输

存储转发传输是指在交换机能够开始向输出链路传输该分组的第一个比特之前，必须接收到整个分组。

排队时延和分组丢失

若链路繁忙，到达分组必须在输出缓存中等待

- 排队时延：分组在队列中等待传输的时间
- 分组丢失：分组在路由器的缓存空间不足时，分组被丢弃

转发表

路由器具有一个转发表，用于将目的地址（或目的地址的一部分）映射成为输出链路

时延

- 节点处理时延 (nodal processing delay)
检查分组首部和决定将该分组导向何处，检查比特差错
- 排队时延 (queuing delay)
在链路上的分组排队等待传输的时间
- 传输时延 (transmission delay)
 L/R , 其中 L 是分组长度, R 是链路传输速率
- 传播时延 (propagation delay)
 d/s , 其中 d 是传播距离, s 是信号传播速率
- 总时延 (total nodal delay)

表示法:

- d_{proc} : 节点处理时延
- d_{queue} : 排队时延
- d_{trans} : 传输时延
- d_{prop} : 传播时延
- $d_{nodal} = d_{proc} + d_{queue} + d_{trans} + d_{prop}$: 总时延

应用程序体系结构

- 客户 - 服务器体系结构:
 - 服务器有固定IP地址, 一直运行
 - 客户端主动连接服务器, 不可直接与其他客户端通信
- P2P(peer)体系结构:
 - 没有一直的服务器
 - 任意端系统之间可以通信
 - 每个端系统既是客户端也是服务器
 - 参与的主机
 - 间歇性连接
 - 可以改变IP地址

进程通性

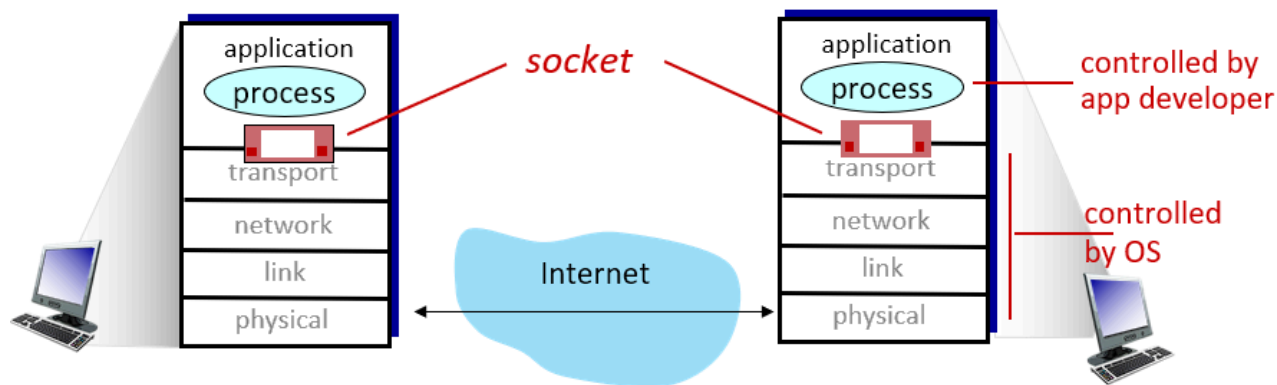
进程process: 在主机上运行的应用程序

- 在同一个主机内, 使用 进程间通信机制通信(操作系统定义)
- 不同主机, 通过交换报文来通信

套接字

进程通过套接字socket与网络通信

通信的双方各有一个套接字, 可以向套接字发送报文, 或从套接字接收报文



进程寻址

通过IP地址和端口号来唯一标识一个进程

对传输服务的要求

- 可靠数据传输
- 吞吐量
- 延迟(时间敏感)
- 安全性

传输层协议

- TCP: 面向连接的, 可靠的, 有序的, 重传
 - 流量控制: 发送方不会淹没接收方

- 拥塞控制: 拥堵时抑制发送方
- UDP: 无连接的, 不可靠的, 无序的, 不重传

TLS

- 提供加密的TCP连接
- 数据完整性
- 端到端的鉴别

HTTP

超文本传输协议, 无状态的, 建立在TCP之上的应用层协议

- 1.0: 一个TCP连接只能传输一个请求, 非持久连接
- 1.1: 一个TCP连接可以传输多个请求, 持久连接
- 2.0: 多路复用, 允许在相同连接中多个请求和回答交错, 将对象分为帧, 帧交错传输, 缓解了**队头阻塞**
- 3.0: 在 UDP上增加了安全性、对象错误控制和拥塞控制

报文格式

- 请求报文
 - 请求行: 方法, URL, 协议版本
 - 首部行
 - 请求主体
- 响应报文
 - 状态行: 协议版本, 状态码, 状态信息描述
 - 首部行
 - 数据

状态码

- 200 OK - 请求成功
- 301 Moved Permanently - 资源的URL已经被更改
- 304 Not Modified - 资源未被修改(条件GET请求)
- 400 Bad Request - 服务器无法解读请求
- 404 Not Found - 服务器无法找到请求的URL

Cookie

维护用户-服务器状态

- 在HTTP响应报文中有一个cookie的首部行
- 在下一个HTTP请求报文含有一个cookie的首部行
- 在用户端系统中保留有一个cookie文件, 由用户的浏览器管理
- 在Web站点有一个后端数据库

SMTP

SMTP 用于从发送方的邮件服务器发送报文到接收方的邮件服务器

1. 用户将邮件发送到**发送方**的邮件服务器
2. 邮件服务器将邮件发送到**接收方**的邮件服务器

SMTP 的特点:

- 采用持续连接
- 多个对象包含在一个多部分报文中
- 要求报文为7位ASCII码
- 使用 CRLF.CRLF 作为报文结束标志
- 推协议

MIME

多用途互联网邮件扩展(MIME), 允许电子邮件携带各种类型的多媒体附件
继续使用SMTP协议, 但增加了邮件主体的结构, 并定义了传输非ASCII码的编码规则

POP3 & IMAP

邮局协议3(POP3), 用于从邮件服务器接收邮件, 功能简单, 不支持多种设备同步和会话状态

- 用户登录
- 下载邮件, 删除标记
- 退出

互联网邮件访问协议(IMAP), 功能更强大

- 保留邮件在服务器上
- 多文件夹
- 可以只下载邮件的头部
- 支持多种设备同步和会话状态

DNS

定义

- 一个由分层的 DNS 服务器实现的分布式数据库
- 使得主机能够查询分布式数据库的应用层协议。

分类

- 根域名服务器
- 顶级域名服务器(TLD)
- 权威域名服务器
- 本地域名服务器

查询方式

- 递归查询: 以自己的名义向其他服务器查询
- 迭代查询: 询问其他服务器, 以自己的名义返回

缓存, 用RR(资源记录)缓存查询结果

TYPE	NAME	VALUE
A	主机名	IP
NS	域名	此域的授权名称服务器的主机名
CNAME	别名	规范名
MX	域名	和NAME关联的邮件服务器名称

P2P

在P2P文件分发中, 每个对等方能够向任何其他对等方重新分发它已经收到的该文件的任何部分, 从而在分发过程中协助该服务器。

视频流

Streaming video = 编码+DASH+播放缓冲

编码

- CBR: 固定码率
- VBR: 码率随时间变化
- 空间编码: 发送颜色和重复的个数

- 时间编码: 仅发送变化的部分

DASH

动态自适应流媒体传输(DASH)是一种流媒体传输技术, 它允许客户端根据当前网络条件和客户端性能选择**不同的视频质量**。

客户端缓冲和播放延迟

补偿网络增加的延迟, 延迟抖动

CDN

内容分发网络(CDN)是一种分布式网络, 用于将内容传输到最终用户, 以提高性能、可靠性和成本效益。
也就是提供了多个资源副本

OTT

OTT(Over-the-Top)服务是指通过互联网提供的各种应用服务

传输层

传输层协议为运行在不同主机上的应用进程之间提供了**端到端的逻辑通信**。

多路复用和多路解复用

- 将传输层报文段中的数据交付到正确的套接字的工作称为**多路分解**
- 在源主机从不同套接字中收集数据块, 并为每个数据块封装上首部信息从而生成报文段, 然后将报文段传递到网络层, 所有这些工作称为**多路复用**

端口号

端口用16 位端口号标志。

端口号只具有**本地意义**, 即端口号只是为了标志本计算机应用层中的各进程。

- 0-1023: 熟知端口号
- 1024-49151: 登记端口号
- 49152-65535: 短暂端口号

TCP

TCP 是一种面向连接的、可靠的、基于字节流的传输层通信协议。

服务器采用4元组(源IP, 源端口, 目的IP, 目的端口)来区分不同的套接字, 实现多路复用和多路分解

UDP

从应用层的进程得到数据之后, 附加上源和目的端口号字段(和其他两个小字段: 长度/校验和), 然后将数据传递给网络层。

- 尽力而为服务
- 可能丢包
- 可能无序交付
- 无连接

优于TCP的地方

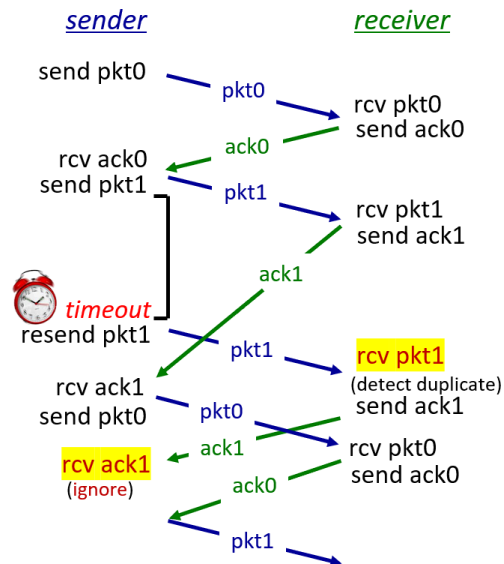
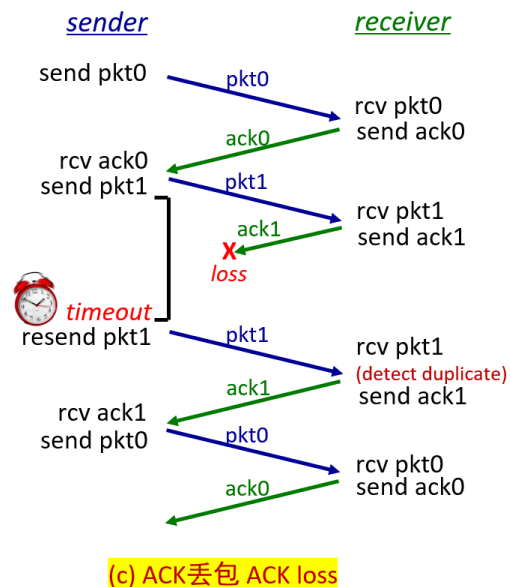
- 无需建立连接(不产生RTT)
- 实时应用容忍丢包, 但要求低延迟
- 无连接状态, 无需维护连接状态
- 分组首部开销小(8字节/ TCP 20字节)

可靠数据传输

RDT(Reliable Data Transfer)协议

- 1.0: 无差错, 无丢包, 无重复
- 2.0: 有限状态机(FSM), 发送ACK和NAK, 会出现冗余(分不清重传还是新的分组)
- 2.1: 对分组做01编号(seq号), 但无法确认ACK是否丢失
- 2.2: 不使用NAK, 发送方收到重复的ACK(带参数), 重传
- 3.0: 超时重传

rdt3.0 动作



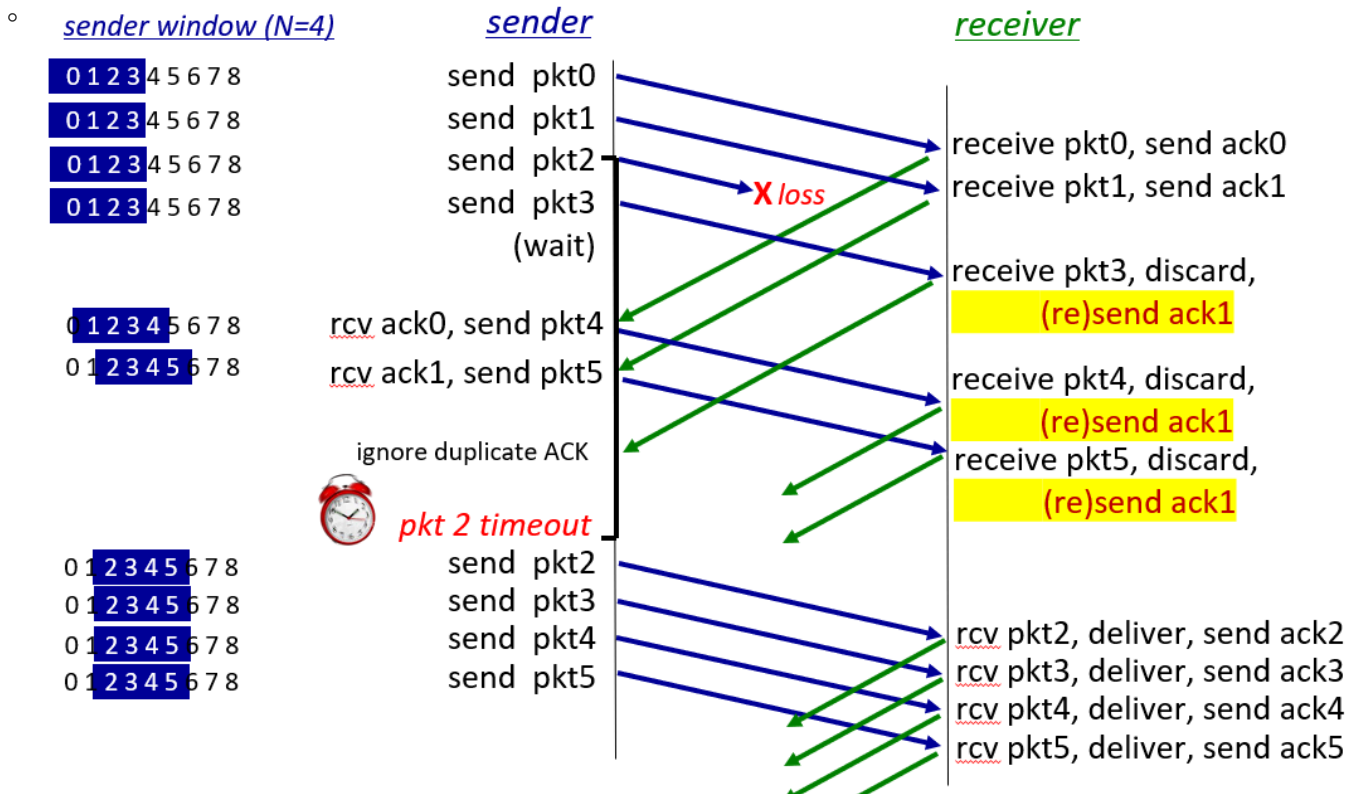
Transport Layer: 3-68

流水线协议: 发送方允许多个未确认的分组在网络中

- 序列号范围增加
- 在发送和接收方都缓存多个分组

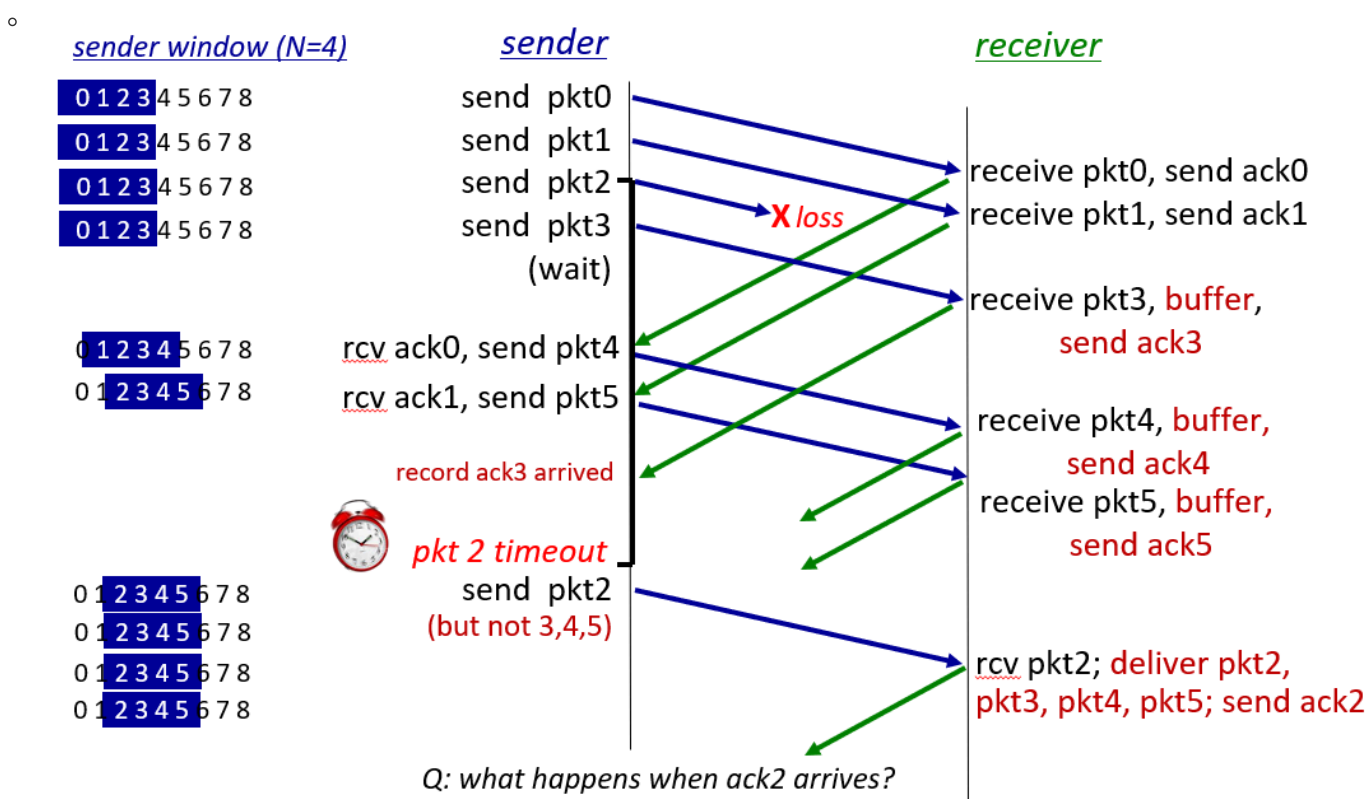
解决差错恢复:

- 回退N步 Go-Back-N(滑动窗口协议)
 - 发送方: 缓存未确认的分组, 重传
 - 接收方: 丢弃失序的分组, 只回复最后一个正确顺序的分组



• 选择性重传协议 Selective Repeat

- 超时了重传丢失的分组(每个分组独立计时)
- 接收方如果收到失序的分组, 缓存, 等待正确的分组到来



选择性重传协议必要条件: 窗口长度必须小于等于序号空间大小的一半
如果窗口比序号小1, 则无法工作

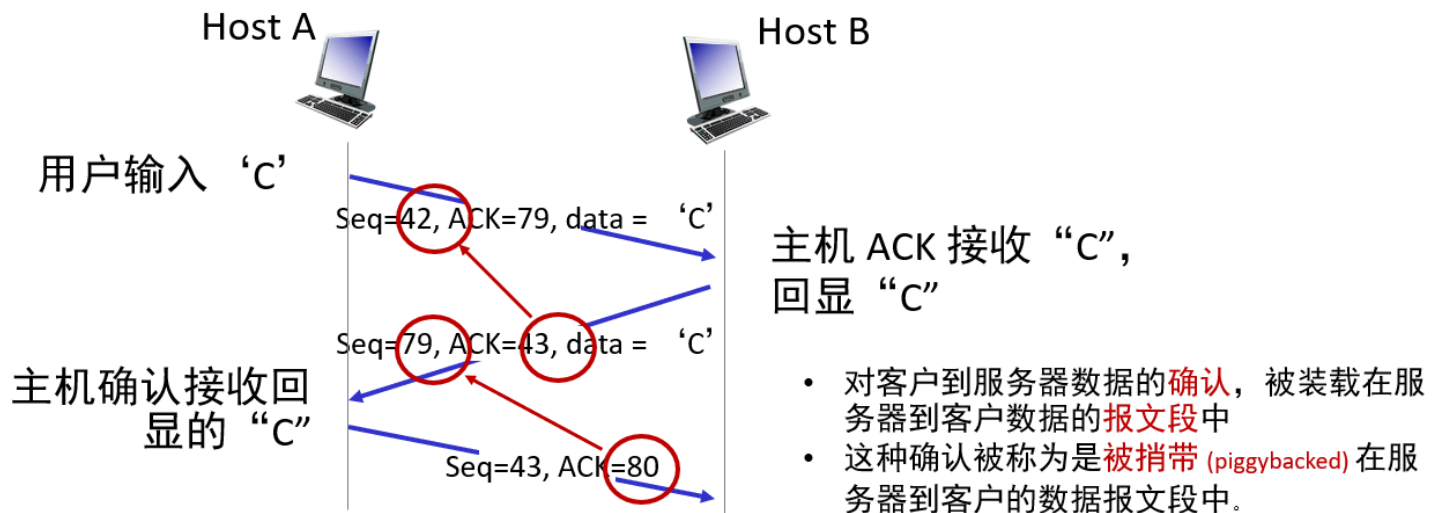
TCP 协议

- 点到点
- 可靠, 有序的字节流
- 全双工数据
- 累积ACKs
- 流水线: TCP拥塞和流量控制设置窗口大小

- 面向连接, 三次握手初始化双方状态
- 流量控制: 发送方不会淹没接收方

Seq: 段数据中第一个字节的字节流数字

ACK: 预期来自另一段的下一个字节的序号



和RDT类似, 最新的按顺序的收到了就发下一个预期的ACK
即便有ACK丢失了, 也会被覆盖

MSS

MSS (Maximum Segment Size)是 TCP 报文段中的数据字段的最大长度。
所以, MSS是“TCP 报文段长度减去 TCP 首部长度”

- MSS 与接收窗口值无关
- MSS较小, 会增加TCP首部的比例, 降低了网络的利用率
- MSS较大, 在IP层分片, 会增加丢包的可能性

MTU是IP链路层的最大传输单元, MSS是TCP层的最大传输单元

MSS应当尽量大到不需要分片, 但不同的IP路径可能有不同的MTU, 会导致分片
建立连接时, 双方写入MSS, 两个方向的MSS可能不同

流量控制

TCP 接收方在 `rwnd` 字段中通告可用的缓冲区间
发送方将未确认的数据量限制为接收的rwnd

当接收方的接收窗口为0时, 发送方用一个持续计数器按固定间隔继续发送只有一个字节数据的报文段(探测报文段), 在确认报文中将包含一个非0的 `rwnd` 值

三次握手

- TCP 连接建立:
 - 客户端向服务器发送TCP 报文段, 不包含应用层数据。
 - SYN报文段: SYN=1
 - 客户随机地选择初始序号 (client_isn), 并放置于该SYN 报文段的序号字段。
- SYN 报文段到达服务器, 服务器为该 TCP 连接分配缓存和变量, 并发送允许连接的报文段
 - SYNACK报文段: SYN =1, ACK=1, 报文段也不包含应用层数据
 - 确认号字段被置为 client_isn + 1
 - 服务器选择自己的初始序号 (server_isn), 并放置到序号字段中
- 对服务器允许连接的报文段进行了确认
 - 将值 server_isn + 1 放置到确认字段。
 - SYN = 0, ACK=1

TCP 3-way handshake

Server 状态

Client 状态

```
clientSocket = socket(AF_INET, SOCK_STREAM)
```

LISTEN

```
clientSocket.connect((serverName,serverPort))
```

SYNSENT

ESTAB

设置seq num, x
发送TCP SYN

SYN=1, Seq=x

SYN=1, Seq=y
ACK=1; ACKnum=x+1

接收SYNACK(x)
表明服务器是活的;
对 SYNACK 发送 ACK;
可以包含数据

ACK=1, ACKnum=y+1

设置seq num, y
发送 TCP SYNACK
确认 SYN

接收 ACK(y)
表明客户端是活的

LISTEN

SYN RCVD

ESTAB

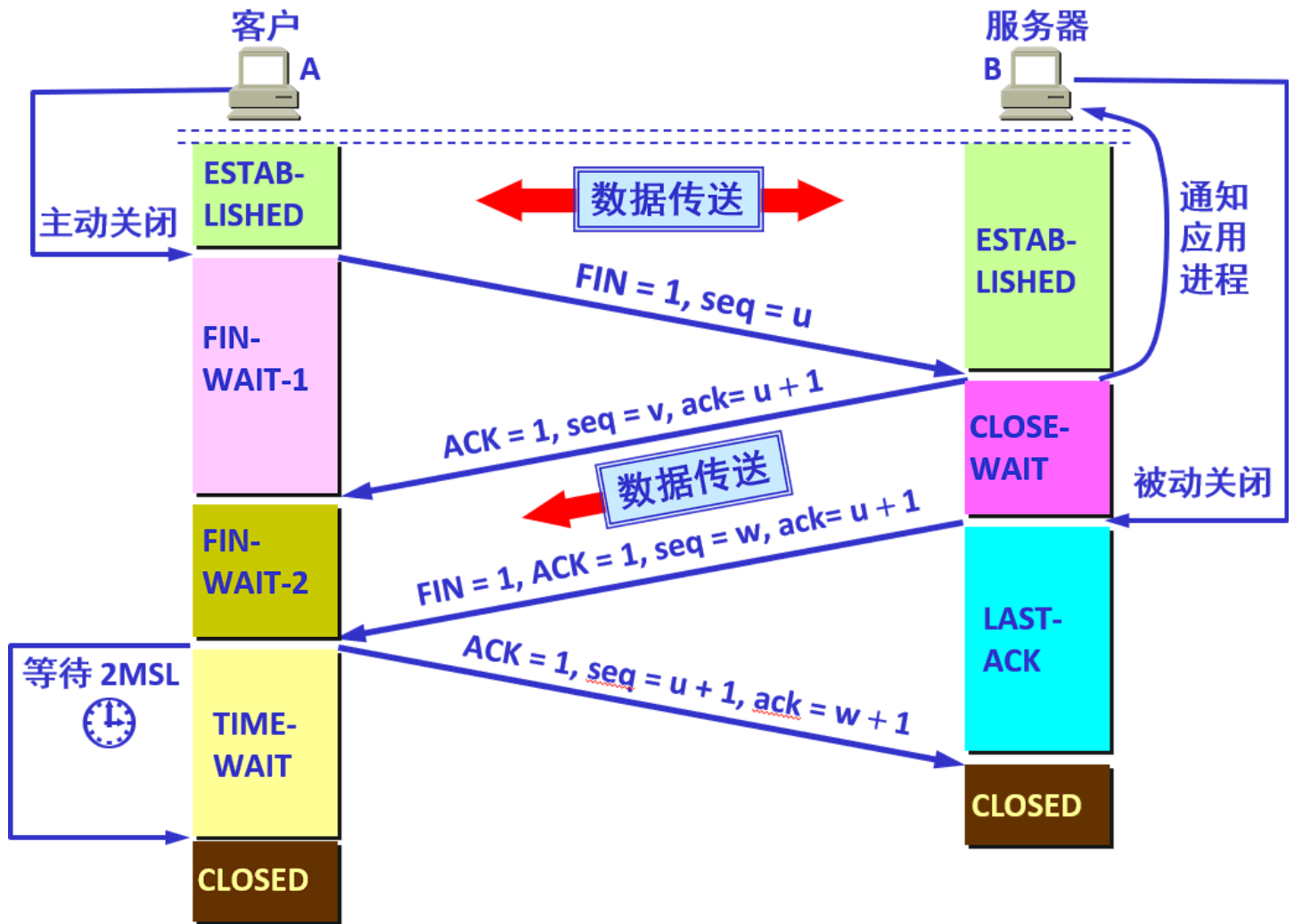
释放连接

主动结束的一方必须等待 2MSL 的时间

- 为了保证 A 发送的最后一个 ACK 报文段能够到达 B。
- 防止“已失效的连接请求报文段”出现在本连接中。

A 在发送完最后一个 ACK 报文段后，再经过时间 2MSL，就可以使本连接持续的时间内所产生的所有报文段，都从网络中消失。就可以使下一个新的连接中不会出现这种旧的连接请求报文段。

TCP 连接必须经过时间 2MSL 后才真正释放掉。



拥塞控制

防止过多的数据注入到网络中，导致网络中的链路不致过载。

拥塞控制的前提：网络能够承受现有的网络负荷

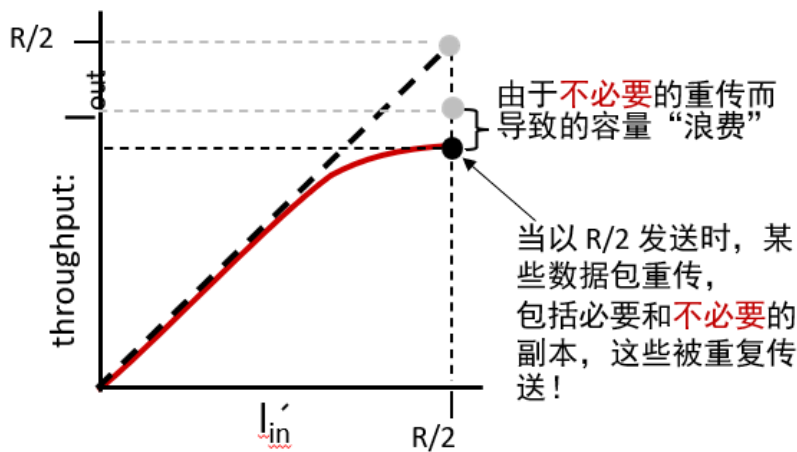
拥塞控制是一个全局性的过程，涉及到所有的主机、路由器和链路。

流量控制：特指点对点通信量的控制，是端到端的问题，接收端控制发送端的发送速率

区别：

- 流量控制：端到端 / 滑动窗口 / 持续计数器
- 拥塞控制：全局网络 / 拥塞窗口 / 重传计数器

发送方发了一个分组，缓存在路由器里，超时之后又发了一个副本
接收方收到了两个副本，因此遇到大延时会产生不必要的重传



结论:

- 吞吐量永远不会超过容量
- 延迟随着容量的接近而增加
- 丢失/重传会降低有效吞吐量
- 不需要的重复发送会进一步降低有效吞吐量
- 下游丢失的数据包浪费了上游的传输容量/缓冲

控制方法:

- 端到端: 将增加的往返时延作为网络拥塞程度增加的指标
- 网络辅助: 路由器向发送方发送拥塞状态的显式反馈信息

AIMD

Additive Increase Multiplicative Decrease

加法增大乘法减小

TCP Reno版本

发送方提高发送率, 直到出现丢包, 降低发送率(增加1个最大报文段的大小/降低一半的发送速率)

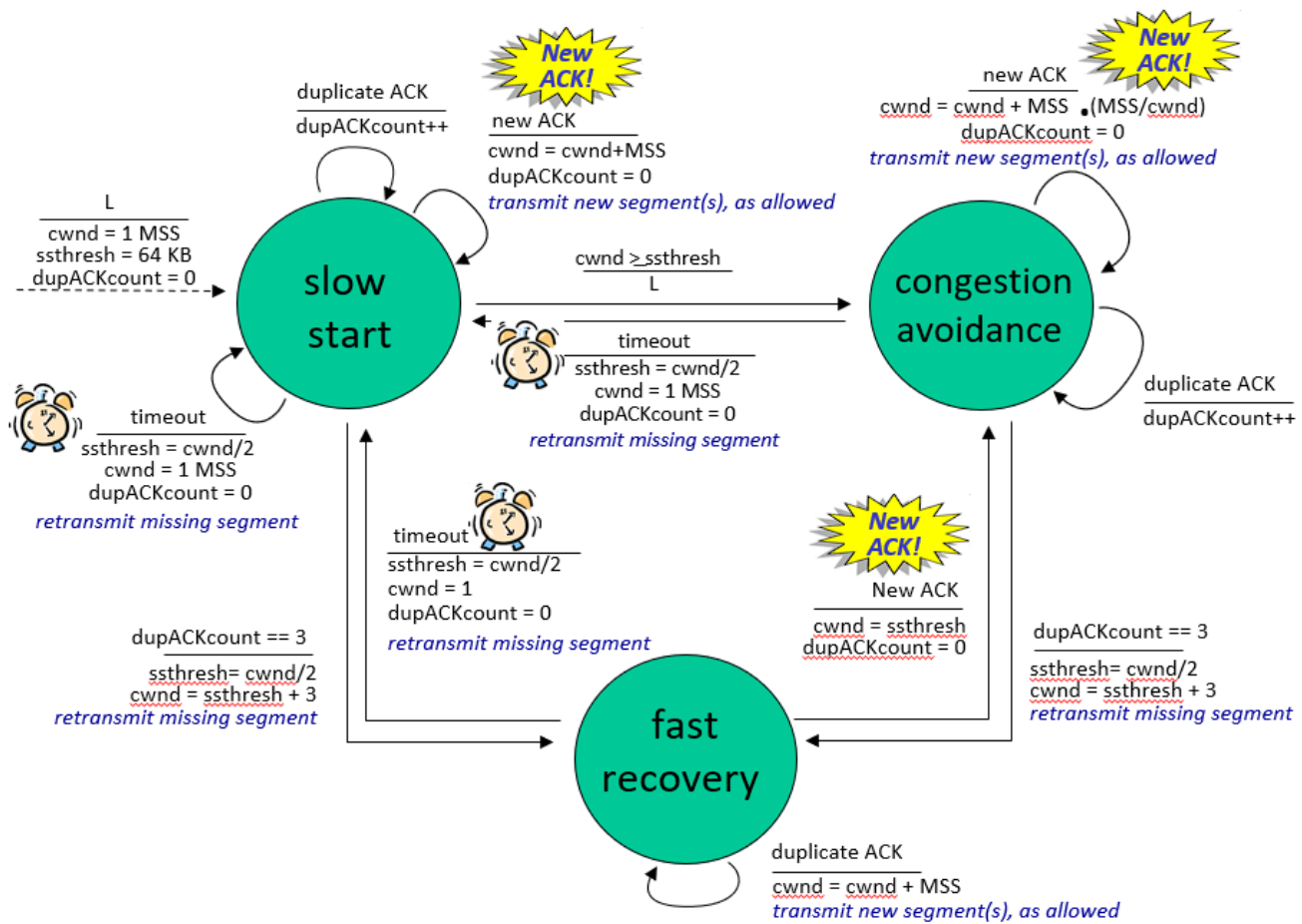
- 三次重复ACK检测到报文丢失, 发送速率减半
- 超时检测到报文丢失, 发送速率减小到1MSS

优点

- 分布式异步算法
- 优化全网拥塞流
- 具有理想稳定性

TCP拥塞控制算法

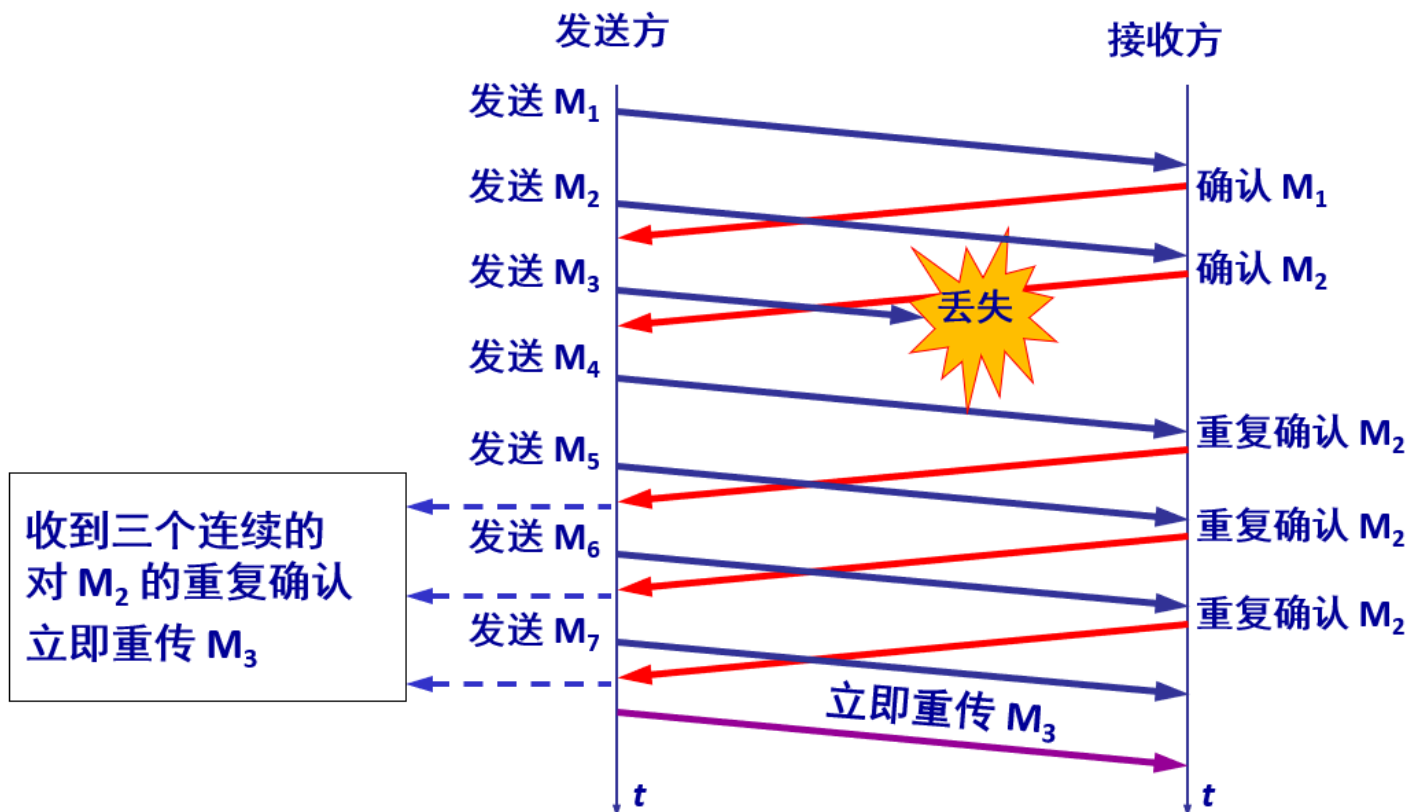
- 慢启动: 初始速率缓慢, 但呈指数增长
 - 出现丢包: $ssthresh = cwnd/2$, $cwnd = 1MSS$, 重新开始慢启动
 - $cwnd$ 增长到 $ssthresh$, 进入线性增长, 转移到拥塞避免模式
 - 出现3次重复ACK, $ssthresh = cwnd/2$, $cwnd = ssthresh + 3MSS$, 进入快速恢复模式
- 拥塞避免: 每次RTT只增加1MSS
- 快速恢复: 对于引起重传的丢包, 每个冗余ACK, $cwnd$ 增加1MSS
 - 收到丢失的ACK后, 进入拥塞避免模式



快重传算法的基本思想是:

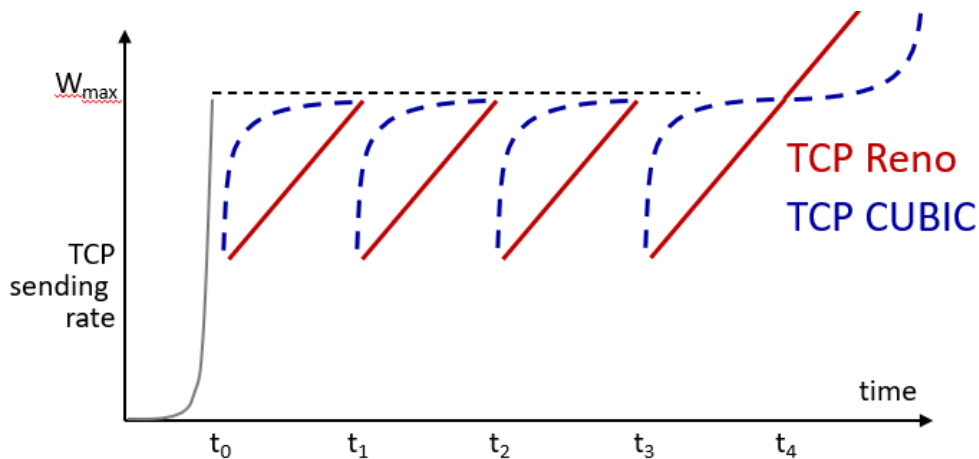
- 接收方收到失序的报文段, 就会立即发送重复的ACK
- 发送方收到3个重复的ACK, 就会立即重传丢失的报文段

注意是收到“3个重复的ACK”, 算上正常的ACK, 一共是4个同一个序号ACK



TCP Cubic

在拥塞避免阶段, 采用更快的增长速率, 随后在接近带宽的时候谨慎地探测带宽



网络层

将段分组从一台发送主机移动到一台接收主机。

- 转发
 - 该路由器必须将该分组移动到适当的输出链路
- 路由选择
 - 当分组从发送方流向接收方时，网络层必须决定这些分组所采用的路由或路径

转发(forwarding)/数据平面: 将分组从输入链路接口转移到适当的输出链路接口的路由器本地动作

路由选择(routing)/控制平面: 确定分组从源到目的地所采用的的端到端路径的网络范围处理过程

两种控制平面:

- 传统方式: 每个路由器有独立路由算法组件，通过控制平面交互
- 软件定义网络(SDN): 远程控制器计算、在路由器中安装转发表

网络服务模型

定义了分组在发送与接收端系统之间的端到端运输特性

- 确保交付
- 具有时延上界
- 有序分组交付
- 确保最小带宽
- 安全性

链路层交换机: 链路层的设备, 依据链路层的地址转发数据帧

路由器: 网络层的设备, 依据网络层的地址转发数据包

路由器工作原理

基于目的地转发

1. 线路端接
2. 数据链路层处理
3. 查找, 转发, 排队
4. 交换结构
5. 排队(缓存管理)
6. 数据链路层处理
7. 线路端接

转发表采用**最长前缀匹配规则**

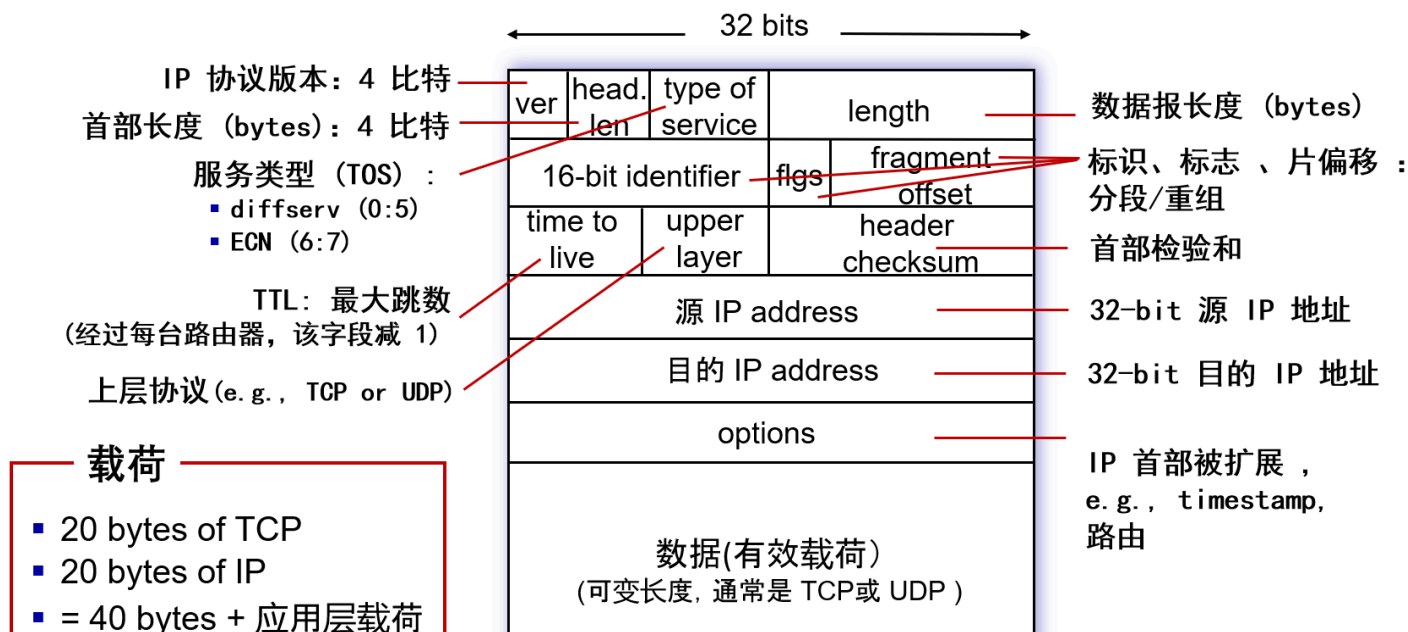
排队

- 输入端: 线路前部阻塞(直行+小转弯, 你明明可以小转弯, 但前面直行的车走不了所以你也走不了)
- 输出端: 没内存了, 丢一个分组(可以发拥塞信号)

分组调度

- **先进先出(FIFO)**, 也称先来先服务(FCFS)
- **优先权排队(PQ)**, 非抢占式优先权排队(下一个有高级的就先处理, 但低级的在处理的时候不能被打断)
- **循环排队规则(round-robin)**, 循环调度器在每个类之间轮流提供服务, 并且要**保持工作排队**(类里没人就跳过)
 - **加权公平排队(Weight Fair Queue)**, 每个类别有一个权重, 每个类至少能保证分配到 $W_i / \sum W_j$ 的带宽

IPv4



首部检验和

- 计算:
 - 两个字节为单位, 以16位为单位, 逐个相加取反码
- 每台路由器都会重新计算检验和(因为TTL会变化)
- IP层的检验和只检查IP首部, TCP/UDP的检验和检查整个数据报

IP分片fragment

不是所有的链路层都支持大的数据帧, 因此需要分片

最大传输单元(MTU)是链路层的最大数据帧大小

每个较小的数据报称为一个**分片**, 用单独的链路层帧传输

分片在进入运输层之前被重组, 在端系统不在路由器

- **源和目的地址**, 都一样
- **标识符**, 用于标识一个数据报的所有分片
- **标志比特**, 除了最后一个分片是0, 其他分片的标志比特都是1
- **偏移字段**, 指明该分片在原始数据报中的位置

IP地址

IP地址: 主机/路由器的接口的标识符, 32位二进制数

子网: 无需通过中间路由器, 即可物理到达彼此的一组设备接口

子网部分: 子网中的设备具有公共的高位地址

主机部分: 剩余的低位地址

子网掩码: IP编码为子网分配的一个地址

223.1.1.0/24 表示前24位是子网掩码, 后8位是主机号

- CIDR
- **CIDR(Classless Inter-Domain Routing)无类别域间路由选择**
外部路由器仅考虑地址前缀
- 路由聚合
使用单个网络前缀通告多个网络
- 分类编址方案:
 - A类: 8位网络号, 24位主机号, 前缀0
 - B类: 16位网络号, 16位主机号, 前缀10
 - C类: 24位网络号, 8位主机号, 前缀110

一个IP地址在整个互联网中是唯一的,但在一个子网中不是唯一的

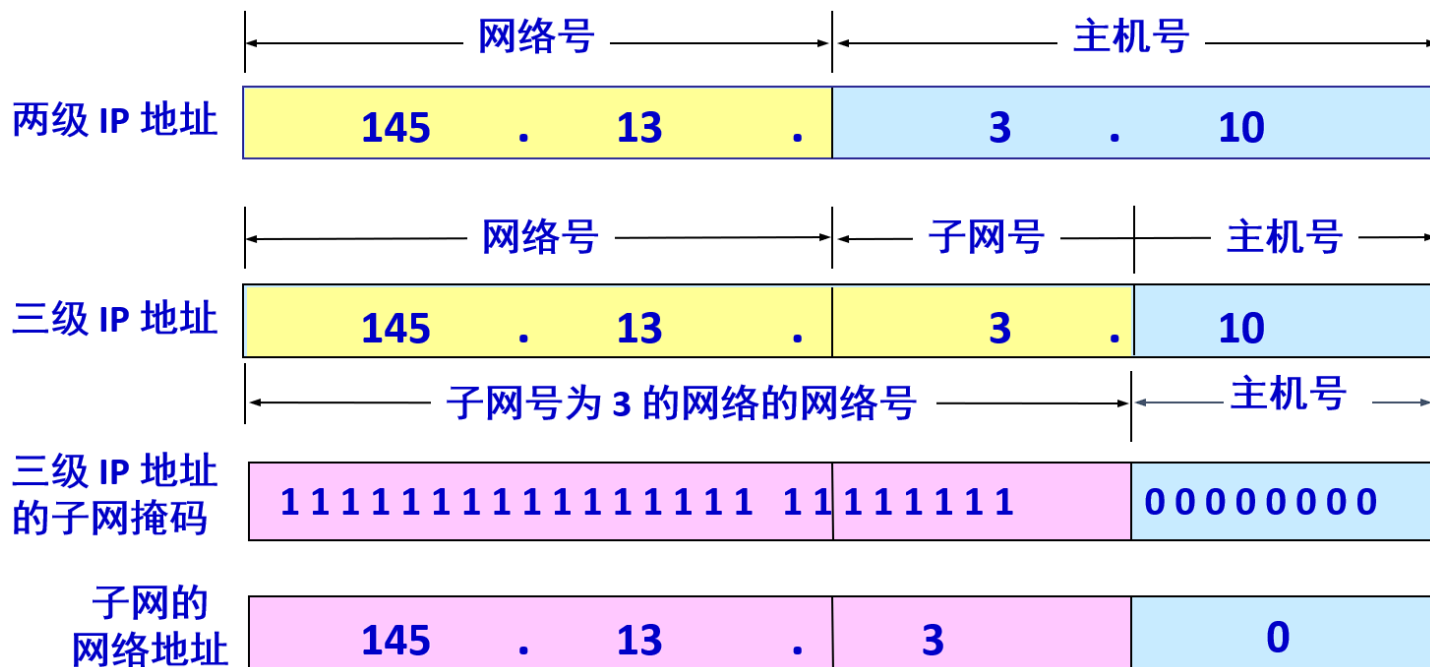
划分子网

从主机号中划分出子网号,用于划分子网

IP地址 = 网络号 + 子网号 + 主机号

子网掩码 = 网络号 + 子网号

IP地址 & 子网掩码 = 网络地址



获取IP地址 DHCP

DHCP(Dynamic Host Configuration Protocol)动态主机配置协议

- 主机加入网络时, 从网络服务器动态获取IP地址
- 即插即用协议, 零配置协议

四个步骤:

1. 主机广播**DHCP发现报文**
 - ip层: 源地址 0.0.0.0 , 广播目的地址 255.255.255.255
2. DHCP服务器广播**DHCP提供报文**
 - 报文包含: 发现报文的事务ID, 提供的IP地址, 子网掩码, IP地址租用时间
3. 主机向DHCP服务器发送**DHCP请求报文响应**
4. DHCP服务器向主机发送**DHCP确认报文响应**

缺陷: 在子网间移动时, 需要重新获取IP地址, 无法维持与远程应用之间的TCP连接

DHCP可以返回的信息:

- 子网上分配的IP地址

- 客户端第一跳路由器的IP地址
- DNS服务器的名称 及 IP地址(注意还不知道MAC地址)
- 网络掩码(指示地址的网络与主机部分)

NAT

网络地址转换(Network Address Translation)

本地网络中所有设备共享一个公共IPv4地址

数据报的NAT IP相同, 但端口号不同

本地所有设备都有"专用"32位IP地址空间:

- 10/8
- 172.16/12
- 192.168/16

优势

- 所有设备只需要一个公共IP地址
- 在不通知外部网络的情况下, 更改本地网络中主机的IP地址
- 在不变更本地网络中设备地址的情况下, 更改ISP
- 安全性: 本地网络内的设备**无法直接寻址**或**外界可见**

实现

- 外出数据包: NAT路由器将本地IP地址替换为公共IP地址(新端口)
- 在NAT转换表中, 记住每个 源IP地址:源端口号 到 NAT IP地址:新端口号 的**转换对**
- 进入数据包: NAT路由器将公共IP地址(新端口)替换为本地IP地址

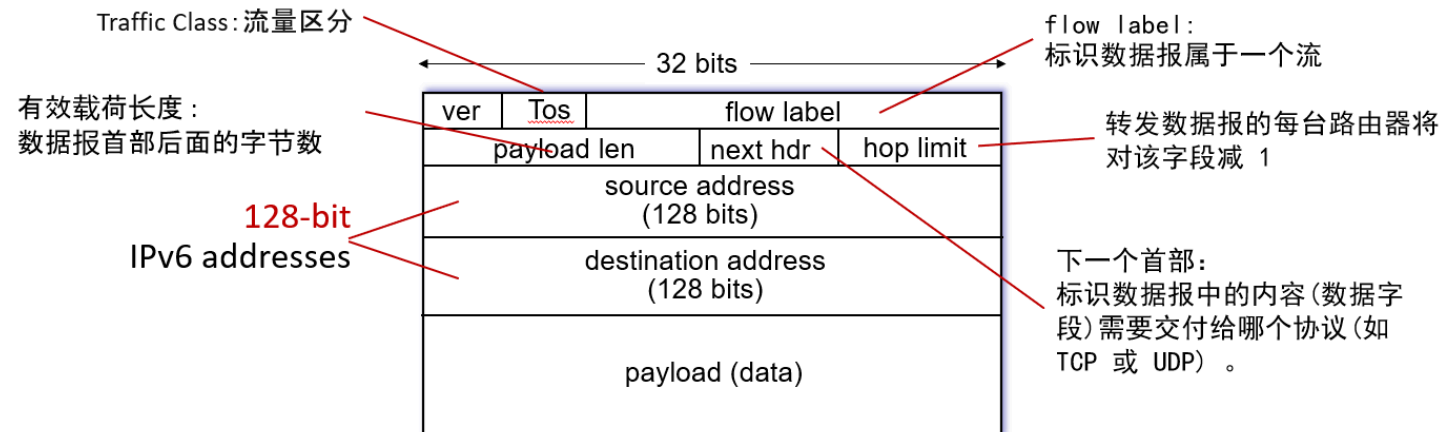
争议

- 路由器应当只处理第三层, 而端口号是第四层的内容
- 违背了端到端原则, 使得网络应用无法直接与对方通信
 - 应当无需借助**中转和变换**, 直接传送到目标主机
 - 外网机器**无法主动**链接到内网机器
 - 一些**应用**设计者需要额外考虑NAT的影响

IPv6

首部格式

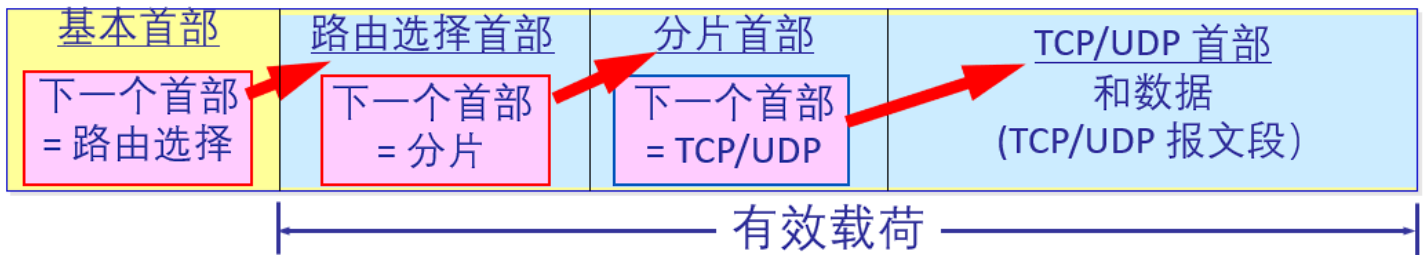
- **固定**40字节头部
- 数据报传输过程中, **不允许分片**



和IPv4的区别

- 无选项: 可以出现在"下一个首部"字段中
- 无首部检验和: 加快处理速度
- 无分片/重组: 由端到端的协议处理

若分组太大, 会被丢弃, 发送方会收到ICMPv6报文, 通知分组太大



隧道

隧道(Tunneling)在IPv4的数据报(载荷)中封装IPv6数据报

"IPv4的隧道连接两个IPv6的网络"

冒号十六进制表示法

每16位用一个十六进制数表示, 用冒号分隔, 128位地址对应8个十六进制数
可以省略前导0, 连续的0用 :: 表示("零压缩")

地址类型

- 链路本地单播地址: FE80::/10 即 1111 1110 10
- 站点本地单播地址: FEC0::/10 即 1111 1110 11
- 兼容IPv4地址: ::/80 即 0000 0000
0:0:0:0:FFFF:IPv4地址
- 全球单播地址: 2000::/3 即 001
 - 第一级: 48位全球路由选择前缀
 - 第二级: 16位子网标识符
 - 第三级: 64位接口标识符

通用转发

匹配加操作: 匹配到达数据包中的位, 并执行相应的操作

基于目标转发: 基于IP地址

- 查找目的IP地址(匹配)
- 将分组发送到有特定输出端口的交换结构(动作)

通用转发: 首部字段决定操作

- 对协议栈多个首部字段"匹配"
- 动作包括
 - 转发到若干输出端口(基于目的地址)
 - 负载均衡
 - 重写首部字段(NAT)
 - 阻挡/丢弃(防火墙)
 - 向特定端口转发(DPI)

OpenFlow

流表: 包含匹配字段和动作字段的条目

一个流表项

- 匹配字段: 包含匹配条件
- 动作字段: 包含要执行的动作
- 状态字段: packet + byte counter

路由协议

按照 传输延迟/经过的站点数量/路由器 等指标, 找到一条从源到目的节点的较好路径

- 最低成本
- 最快
- 最不拥塞

最优化原则: 汇集树

- 一个节点到所有其他节点的最短路径形成一棵树
- 路由算法为所有路由器找到并使用汇集树

路由选择算法原则

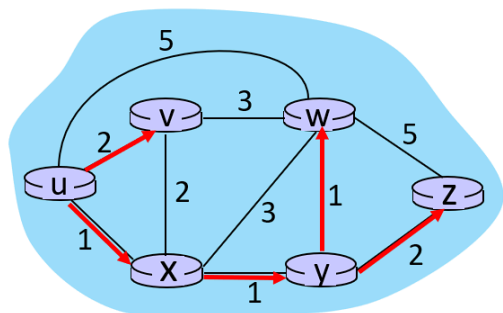
- 正确性/完整性
- 简单性
- 健壮性
- 稳定性
- 公平性
- 最优性

链路状态路由选择算法

所有节点都有相同的网络拓扑图和链路代价

用dijkstra算法为每个节点找到最短路径

Step	N'	^v $D(v), p(v)$	^w $D(w), p(w)$	^x $D(x), p(x)$	^y $D(y), p(y)$	^z $D(z), p(z)$
0	u	2, u	5, u	1, u	∞	∞
1	ux	2, u	4, x		2, x	∞
2	uxy	2, u	3, y			4, y
3	uxyv		3, y			4, y
4	uxyvw					4, y
5	uxyvwz					



find a not in N' such that $D(a)$ is a minimum
add a to N'
update $D(b)$ for all b adjacent to a and not in N' :
 $D(b) = \min (D(b), D(a) + c_{a,b})$

如果链路代价取决于流量, 会发生路由振荡, 解决方法:

- 强制链路开销不依赖于流量(不可接受)
- 并非所有路由器都同时运行LS算法
 - 算法执行时间不同, 会出现自同步
 - 通过随机化, 使得不同路由器在不同时间运行算法

距离向量路由选择算法

迭代 / 异步 / 分布式

- 分布: 每个节点从 直接相邻节点 收集信息, 并将计算结果发送给相邻节点
- 异步: 不要求步调一致
- 迭代: 重复执行, 直到无需更新

链路代价变化

- 节点检测到链路代价变化
- 更新路由信息, 重新计算本地DV
- 若发生变化, 通知相邻节点

好消息传播快, 坏消息传播慢

两者比较

- 报文复杂度: $DV > LS$
- 收敛速度: $LS > DV$
- 路由器故障: $LS > DV$ (错误会扩散)

可扩展路由

自治系统(AS): 一组路由器和网络, 受同一组织控制
同一AS内部的路由器必须运行相同的 **域内协议**

AS 域内路由协议

- RIP: 距离向量(不再广泛使用)
- EIGRP: 增强的距离向量(思科专有)
- OSPF: 开放最短路径优先

OSPF

OSPF(Open Shortest Path First)开放最短路径优先
在一个AS内部, 为每个节点找到最短路径

链路状态

- 路由器将链路状态信息发送给所有其他路由器(直接走IP)
- LS分组在网络中传播, 通过洪泛算法
- 每个路由器都有完整的网络拓扑图

两级层次结构: 本地 / 骨干

- 每个节点知道本地网络的拓扑
- 对于其他区域, 只知道 **区域边界路由器**

分工

- 区域边界路由器: 负责向其他区域边界路由器**通告**
- 本地路由器: 仅在区域内泛洪LS, 计算区域内的路由
- 边界路由器: 通告到其他AS
- 骨干路由器: 在骨干区域内运行OSPF路由

BGP

BGP(Border Gateway Protocol)边界网关协议
将各个AS连接起来, 允许子网向其余互联网通告其存在

提供给AS的协议

- **eBGP**: 从相邻AS获得子网可达信息
- **iBGP**: 将子网可达信息传播到本地AS内部所有路由器
- 根据子网可达信息和策略, 决定到达子网的"好"路径

热土豆路由

热土豆路由(Hot Potato Routing) 选择最小域内代价的本地网关作为前往目的地的出口
检查相邻节点, 选离目标最近的节点, 传递给它

如果都是同样的距离, 随机选择一个

不考虑 **域间代价**

核心思想: 尽快将数据包传递给下一个节点

BGP 选择标准

- 最高本地偏好(取决于AS管理员)
- 选择最短AS-PATH路由
- 用热土豆路由选择最靠近NEXT-HOP的路由

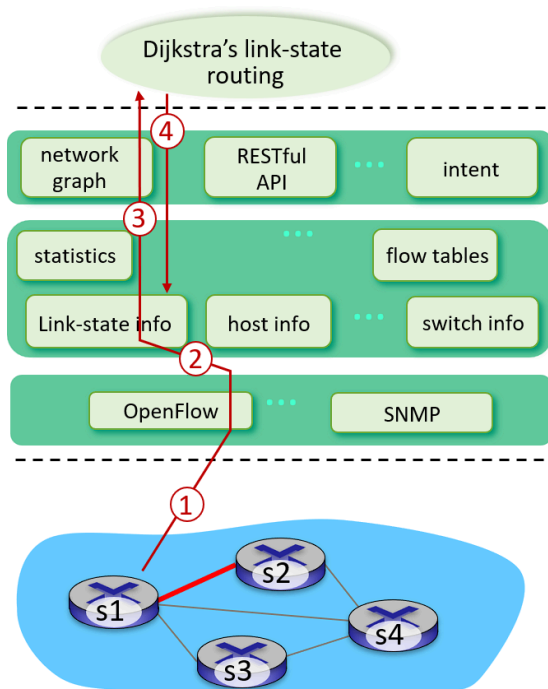
SDN

软件定义网络(Software Defined Network)能通过编程方式控制网络设备, 使网络更加灵活

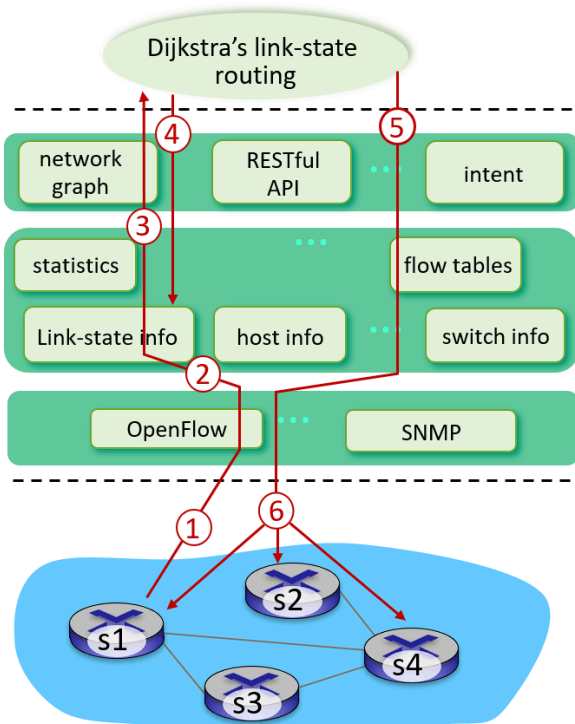
4个关键特征

1. 通用的"基于流"转发
 2. 控制与数据平面分离
 3. 数据平面交换机外部的网络控制功能
 4. 可编程控制应用
- 数据平面交换机
 - 快速/简单 交换机(硬件)
 - 在控制器的指导下, 计算/安装流表
 - 基于表的转发(OpenFlow)
 - SDN控制器
 - "北向"接口: 与网络控制应用通信
 - "南向"接口: 与网络交换机交互
 - 分布式系统实现
 - 网络控制应用程序
 - 控制的大脑
 - 可由第三方提供

SDN: 控制/数据平面交互: 例



- ① S1, 遇到**链路故障**, 使用 OpenFlow 端口状态消息**通知**控制器
- ② SDN 控制器**接收** OpenFlow 报文, **更新**链路状态信息
- ③ 在链路状态更改时**调用**Dijkstra 路由算法应用程序
- ④ Dijkstra路由算法访问网络的图信息、控制器中的链路状态信息, **计算**新路由



⑤ 链路状态路由应用与 SDN 控制器中的**流表计算组件**交互，该组件**计算所需的新流表**

⑥ 控制器使用 OpenFlow 在需要更新的交换机中**安装新表**

ICMP

Internet控制消息协议(Internet Control Message Protocol)

主机和路由器 用与传递 **网络级信息**

- 错误报告
- 回显请求/回显应答

traceroute: 多组UDP报文段, 每组TTL递增, 用于检测路由器

第n组报文段, TTL=n, 会到达第n个路由器, 路由器丢弃报文段, 并发送给源主机一个ICMP告警报文

网络管理

说白了, 管理各种资源和设备, 以合理的成本满足性能/服务需求

组成部分

- 管理服务器
- 网络管理协议
- 设备
- 数据

管理方法

- SSH: 单个设备直接操作
- SNMP/MIB: 网络管理协议 / 设置设备数据
- NETCONF/YANG: 更抽象/更全网

SNMP

传达MIB信息的两种方式:

- 请求响应: SNMP管理服务器向代理发送请求, 代理执行请求并返回结果
- 陷阱: 代理向管理服务器发非请求的信息(陷阱报文), 用于通知管理服务器

链路层

- **节点**: 任意运行链路层协议的设备
- **链路**: 两个相邻节点之间的通信信道

链路层提供:

- **成帧**: 将数据报封装为帧
- **链路接入**: 媒体访问控制(MAC)规定了帧在链路上的传输规则
- **可靠交付**: 通过确认和重传实现
- **差错检测和纠正**: 一般用硬件实现

半双工通信, 不能同时传输

链路层主体在网卡(NIC)/芯片实现

接口通信基本原理

- 发送方: 数据报封装为帧, 添加错误检查位
- 接收方: 查找错误, 提取数据报

差错检测

EDC(Error Detection and Correction)差错检测和纠正

奇偶校验

单比特奇偶校验, 如果数据中有奇数个1:

- 偶校验: 校验位为1, 使得1的出现偶数次
- 奇校验: 校验位为0, 使得1的出现奇数次

二维位奇偶校验, 横纵都出错可以定位到具体的位

单比特只 **检测** 错误,

二维位可以 **检测并纠正** 错误

Internet校验和

将 UDP报文段内容(包括 UDP首部字段 和 IP地址) 每16位相加, 取反码

接收方相加为0, 说明没有出错

CRC

循环冗余校验(Cyclic Redundancy Check)

- D 数据位(二进制)
- d 数据位数
- G 位模式(生成器), $r+1$ 位
- R 校验位(余数), r 位

如果正确, $\langle D, R \rangle$ 可以被 G 整除

求取 R 的方法

- 将 D 左移 r 位
- 长除法, 除数为 G , 余数为 R

多路接入链路&协议

节点如果同时传输两个以上的帧, 会发生 **冲突/碰撞**

那之后接受方都会收到错误的帧

需要 共享信道的分布算法 来确定合适的发送时间

理想的多路接入链路

- 多路访问信道速率: R
- M 个用户, 每个用户平均发送速率: R/M

MAC(Multiple Access Control)多路访问控制协议

- 信道划分
- 随机接入
- 轮流

信道划分

TDMA(Time Division Multiple Access)时分多路访问

- 在每个时间帧中, 将时间划分为 N 个 **时隙**
- 每个站点在每个轮次中获得固定长度的时隙(数据包传输的时间)
- 未使用的时隙被浪费(空闲状态)

电话网络的PCM基于TDMA

PCM(Pulse Code Modulation)脉冲编码调制

FDMA(Frequency Division Multiple Access)频分多路访问

- 信道频谱划分为 N 个 **频段**
- 每个站点分配一个固定频段

频分复用的用户在同样的时间里 占用不同的带宽资源

CDMA(Code Division Multiple Access)码分多址

- 各个用户使用不同的码型, 彼此不干扰

很强的抗干扰能力, 频谱像白噪声, 不易被敌人发现

- 每个比特时间划分为 m 个短的时间间隔, 称为 **码片**
- 每个用户都有一个唯一 m bit 的码片序列
 - 如果要发送1, 就发送码片序列
 - 如果要发送0, 就发送码片序列的反码

原本数据率为 b bit/s, 现在变为 $m * b$ bit/s

占用的频带宽度为 m 倍

每个码片序列必须

- 互不相同
- 相互正交

随机接入

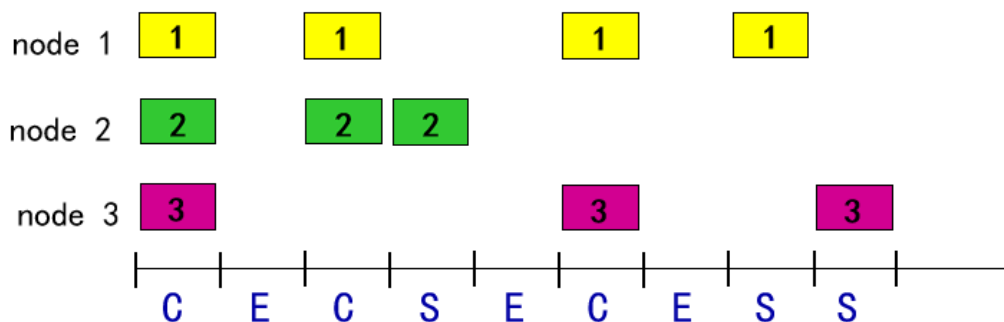
直接发, 然后检测冲突, 并从中恢复

ALOHA(Slotted ALOHA) 时隙ALOHA

- 所有帧大小相同: L bit
- 时间划分为大小相等的时隙: L/R 秒(传输一帧的时间)
- 节点同步, 都只在时隙开始时发送

若无冲突: 接着发送新的帧

若有冲突: 改用概率 p 重新发送, 直到成功



C: 冲突时隙
S: 成功时隙
E: 空闲时隙

- 优点
 - 单个活跃节点可以 **信道全速率**
 - 简单
- 缺点
 - 多个活跃节点时, 会有 **冲突**
 - 空闲时隙浪费

纯ALOHA

无需同步

- 当一帧首次到达, 节点立刻将其完整传输进广播信道
- 若有冲突, 立即以p的概率重传, 否则等待下一帧

效率 只有 $1/2e \approx 18.4\%$

CSMA

CSMA(Carrier Sense Multiple Access)载波监听多路访问

- 载波监听: 在发送前检测信道是否空闲, 空闲则发送整个帧
- 碰撞检测: 在发送过程中检测碰撞, 碰撞则停止发送
- 退避算法: 等待一段时间后重试

Ethernet CSMA/CD 算法

1. NIC(网卡) 从网络层接收数据
2. 网卡**检测**信道
 - 空闲: 启动帧传输
 - 忙碌: 等待
3. 如果传输过程中不发生冲突, 则成功
4. 若检测到另一个传输, 中止传输, 发送**干扰信号**
5. 中止传输后, 进入 **二进制指数退避算法**
 - 第 m 次冲突, 从 $\{0, 1, 2, \dots, 2^m-1\}$ 中随机选择一个数 k , 作为等待时间
 - 等待 $k \cdot 512$ 比特时间, 重新开始步骤2

效率定义为: 帧在无冲突的情况下长期运行中所占的时间份额

- 两节点间传播所需最大时间: T_{prop}
- 传输一个最大长度的帧的时间: T_{trans}

效率为: $efficiency = \frac{1}{1+5\frac{T_{prop}}{T_{trans}}}$

轮流

轮流 Taking Turns Mac协议

- 信道划分: 高负载高效/公平; 低负载效率低
- 随机接入: 低负载高效, 充分利用; 高负载碰撞开销
- 轮流: 两全其美
- 轮询:
 - 主节点依次 **邀请** 节点发送
 - 缺点: **延迟 / 单点故障**
- 令牌传递:
 - 有东西要发就持有令牌, 发送最大数目的帧, 然后传递给下一个节点
 - 不需要发送帧的节点 直接传递令牌
 - 缺点: 令牌丢失 / **单点故障**

MAC地址

用于“本地”获取从一个接口到另一个物理接口的数据帧

- 48位二进制数
- 通常以十六进制表示
- 2个4位数字, 6组, - 分隔

局域网每个接口具有

- 唯一的48位MAC地址
- 本地唯一的32位IP地址

MAC具有"便携性": 换LAN, MAC不变

ARP

ARP(Address Resolution Protocol)地址解析协议

- 用于将IP地址解析为MAC地址
- arp表: <IP地址, MAC地址, TTL>
- TTL: 地址映射遗忘时间

首次发送, 请求流程:

1. 广播ARP请求, MAC填 FF-FF-FF-FF-FF-FF
2. 接收方响应ARP应答, 包含自己的MAC地址
3. 发送方收到应答, 添加条目到ARP表

以太网

主导有线局域网技术

物理拓扑:

- 总线: 同一碰撞域
- 交换式: 中心有链路层交换机, 节点间不相互冲突

集线器(hub)只是中继器, 重新放大信号; 如果同时两个接口发送, 会发生冲突
交换机(switch)是"无碰撞"的, 存储转发分组

以太网帧结构



前同步码:

- 用于同步接收方、发送方时钟速率
- 前 7 字节用于“唤醒”接收适配器
- 7 个字节的10101010, 后跟一个字节的10101011

数据字段 (46 -1500 字节)。

- 这个字段承载了 IP 数据报。
- 以太网的最大传输单元 (MTU) 是 1500 字节。
- 数据字段的最小长度是 46 字节。

■ 地址：6 字节的源MAC 地址、目标 MAC 地址

- 如果适配器接收到具有**匹配目标地址、或广播地址**（例如 ARP 数据包）的帧，它将帧中的数据传递到网络层协议
- 否则，适配器将丢弃帧

■ 类型：表示更高层协议

- 主要是IP，但其他可能，例如Novell IPX，AppleTalk
- 用于在接收方解复用

■ CRC：接收器的循环冗余校验

- 检测到错误：帧被丢弃
- 无连接: 没握手
- 不可靠: 无确认
- MAC协议: 二进制指数退避算法的CSMA/CD

交换机

交换机是一个 **链路层设备**

- **存储 & 转发** 以太网帧
- **检查** MAC地址, 使用CSMA/CD算法
- "透明": 主机不知道交换机的存在
- 即插即用: 无需配置

每个传入链路使用以太网协议

每个链接都有自己的冲突域

A->A' 和 B->B' 之间的传输不会干扰
但是 A->C 和 B->C 之间的传输会干扰

自学习

1. 收到帧: 从传入LAN帧记录 **发送方 & 位置** 对
2. 使用目的MAC地址的检错表, 确定 **输出端口**
 - 如果目的MAC是到达的端口, 则丢弃
 - 否则转发到对应端口
3. 如果目的MAC地址不在表中, 则 **泛洪**(除了发送方, 在所有端口广播)

对比路由器

- 都是存储转发
 - 路由器: 网络层
 - 交换机: 链路层
- 都有转发表
 - 路由器: IP地址, 路由算法
 - 交换机: MAC地址, 泛洪/自学习

VLAN

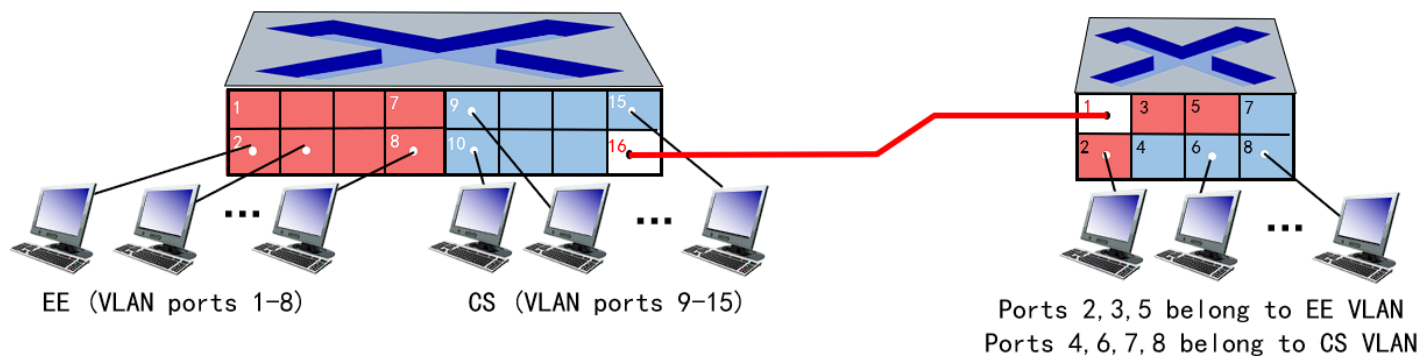
物理上链接A的交换机, 但在逻辑上链接到B的交换机

VLAN(Virtual Local Area Network)虚拟局域网

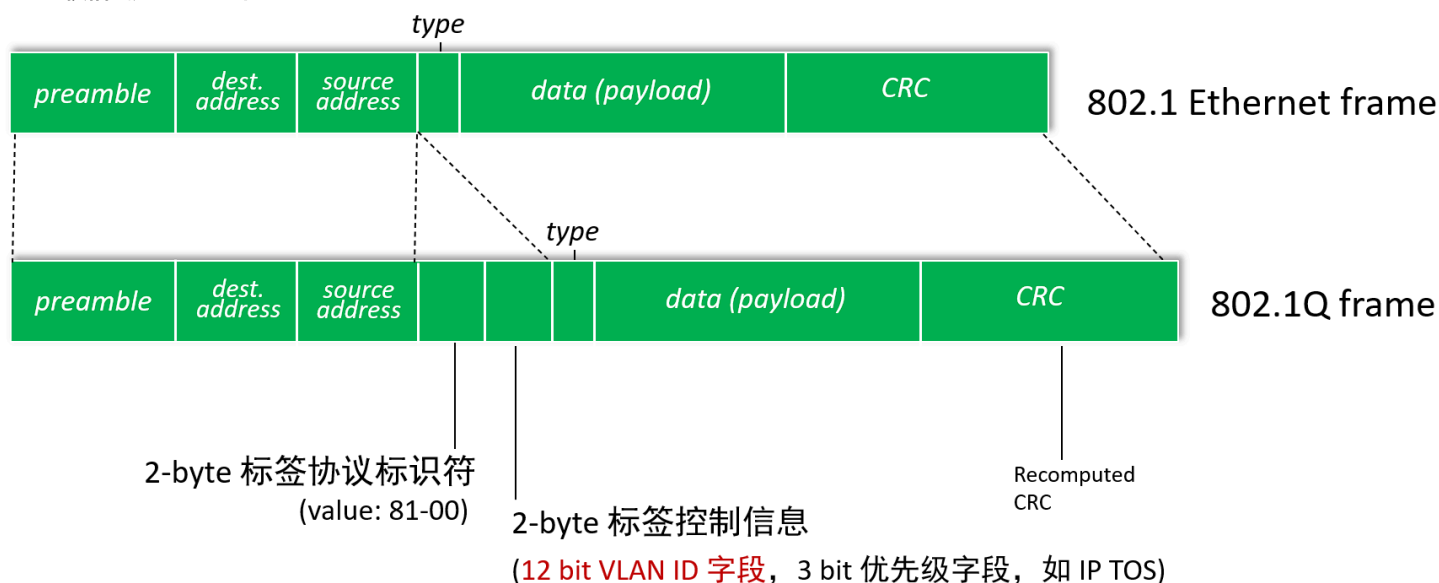
可以将支持VLAN的交换机配置为: 在单个物理LAN基础架构上定义多个虚拟LAN

- 流量隔离: 进出VLAN的流量不会进入其他VLAN
- 动态成员: 在VLAN之间动态分配端口
- VLAN之间的转发: 组合路由器和交换机

跨越多个交换机



VLAN帧格式加入了4字节的标签



加进首部的 4 字节 VLAN 标签 (VLAN tag)

无线链路

- 信号强度降低
- 来自其他设备的干扰
- 多路径传播

SNR(Signal-to-Noise Ratio)信噪比
信噪比越大, 更容易从噪声中恢复信号

WLAN在不发生干扰的情况下, 允许多个设备同时发送
b->a 其实不影响 c->d, 即 **暴露站问题**

码分多址(CDMA)在无线链路中很重要
允许多个用户共存, 并以最小的干扰同时传输(要求正交)

- 编码: 内积, 数据为1, 乘以码片序列; 数据为0, 乘以码片序列的反码
- 解码: 内积, 取均值, 正数为1, 负数为0

802.11 WLAN

AP(Access Point)接入点, 基站
一个主机可以选择周围的一个AP连接, 称为 **关联**

- 被动扫描
 - i. AP广播 **信标帧**
 - ii. 主机接收到信标帧, 选择一个AP, 发送 **关联请求帧**
 - iii. 所选AP发送 **关联响应帧**

- 主动扫描
 - i. 主机发送 **探测请求帧**
 - ii. AP发送 **探测响应帧**
 - iii. **关联请求帧**
 - iv. **关联响应帧**

801.11采用的是 **CSMA/CA**协议, 避免碰撞

- 传输前检测, 不与另一个传输中的帧冲突
- 无碰撞检测, 因为强传输, 弱接收

区别于以太网的CSMA/CD

以太网: 可以检测碰撞

802.11: 无法检测碰撞, 只能避免碰撞

采用 **链路层确认**

- 目的站点收到帧后, 发送 **ACK帧**(中间等一个 **SIFS**时间)
- 发送站点收到ACK帧后, 才认为帧已成功传输
- 收不到就重传, 有最大重传次数, 超过了就丢弃

SIFS(Short Interframe Space)短间隔时间, 用来分隔开属于一次对话的各个帧, 一个站要在此期间, 从发送方切换到接收方

DIFS(Distributed Interframe Space)分布间隔时间, 用来分隔不同对话的帧

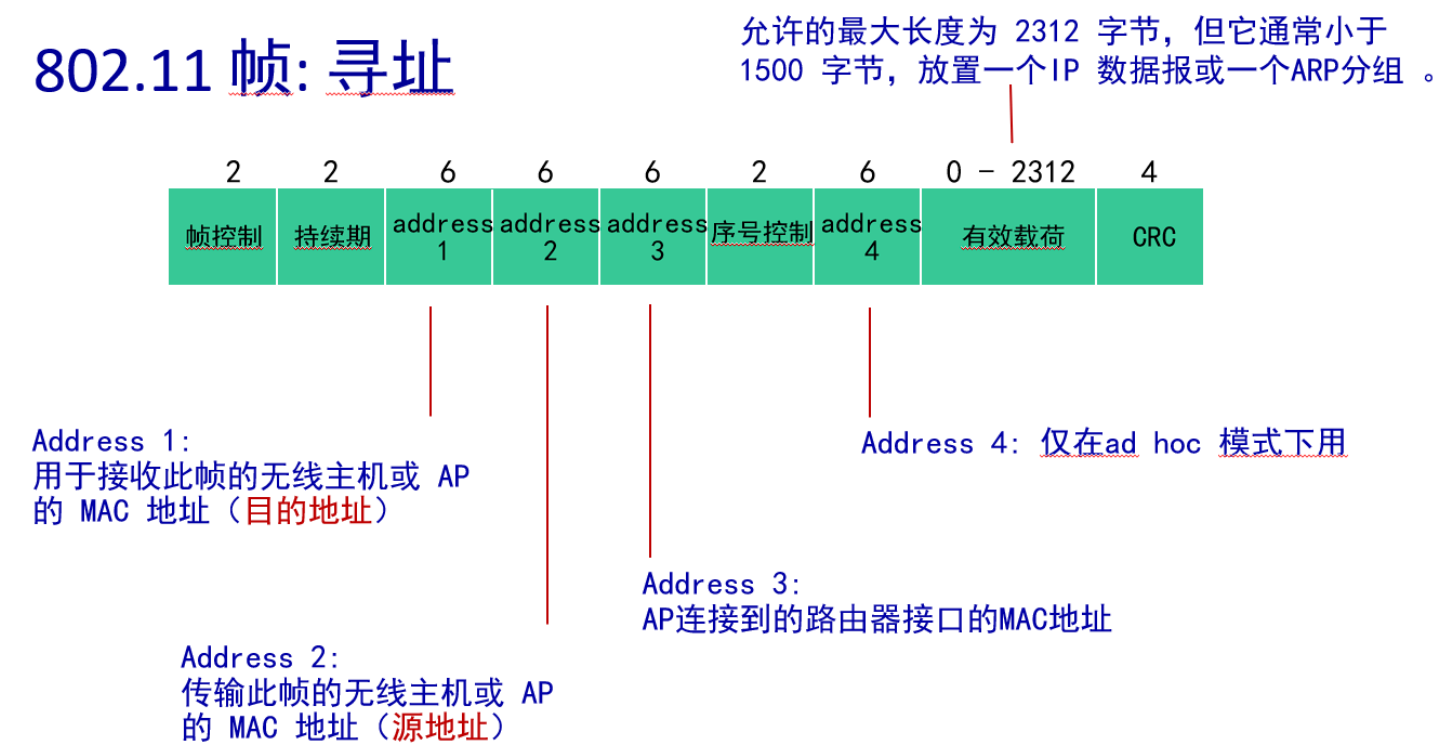
碰撞避免: RTS/CTS

RTS(Request to Send)请求发送, CTS(Clear to Send)允许发送

1. 发送方用 CSMA 发送 RTS帧
2. AP 广播 CTS帧
 - 明确许可发送方
 - 指示其他站点暂停发送
3. 所有节点都听到 CTS帧,
 - 发送方发送数据帧
 - 其他站点推迟发送

如果RTS撞了, 会等待一段时间再重试

802.11帧结构



802.11 同一子网内移动性

- H1逐步远离AP1, 解除与AP1的关联, 关联AP2, 保持IP地址, 维持TCP连接
- 交换机会将帧转发到新的AP(自学习)

802.11 高级

- 速率适应
 - SNR低, 距离越远, 误码率越高
 - 过高时, 切换到较低的速率 & 较低的误码率
- 功率管理
 - 使得打开电路的时间最小
 - 节点向AP发送 **请求睡眠帧**, 直到下一个信标帧

计算

传输时间

- 分组长度: L 比特
- 链路传输速率: R 比特/秒
- 传输一个分组的时间: L/R 秒
- P 个分组经过 N 个路由器的**传输时延**: $(P + N - 1) \frac{L}{R}$ 秒
- 物理距离: d 米
- 信号传播速率: s 米/秒
- 传播一个分组的**传播时延**: d/s 秒
- 分组到达队列的平均速率: a 分组/秒
- **流量强度**: $\frac{La}{R}$

缓存

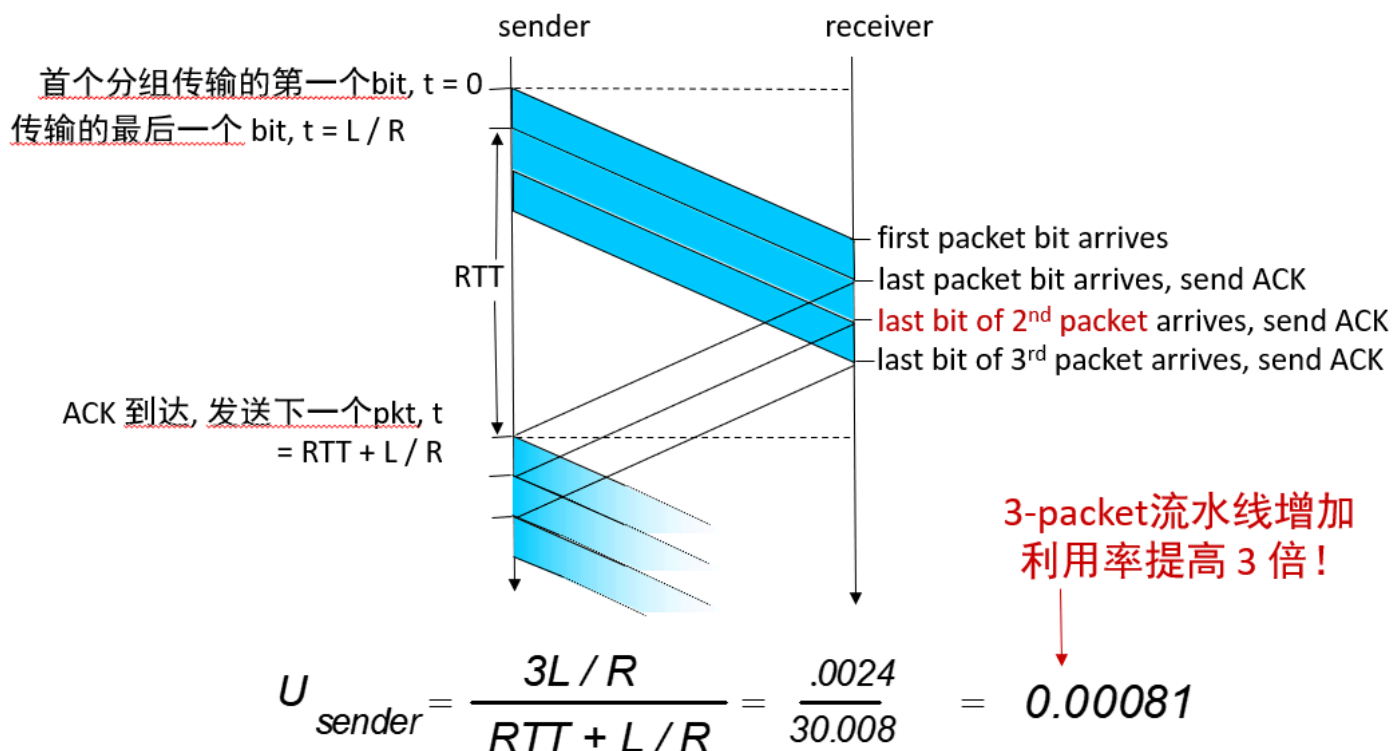
- 已知条件
 - 访问链路速率: $R1$ Mbps
 - 局域网速率: $R2$ Mbps
 - 机构路由器到源服务器的链路的RTT: RTT_1 秒
 - Web页面大小: L 比特
 - 浏览器请求Web页面的平均速率: a 请求/秒
- 现状计算
 - 浏览器请求平均数据速率: La
 - 局域网流量强度: $\frac{La}{R}$
 - 链路流量强度: $\frac{La}{R}$
- 加装局域网Web缓存器
 - 缓存命中率: h
 - 链路流量强度: $\frac{Lah}{R}$

P2P计算

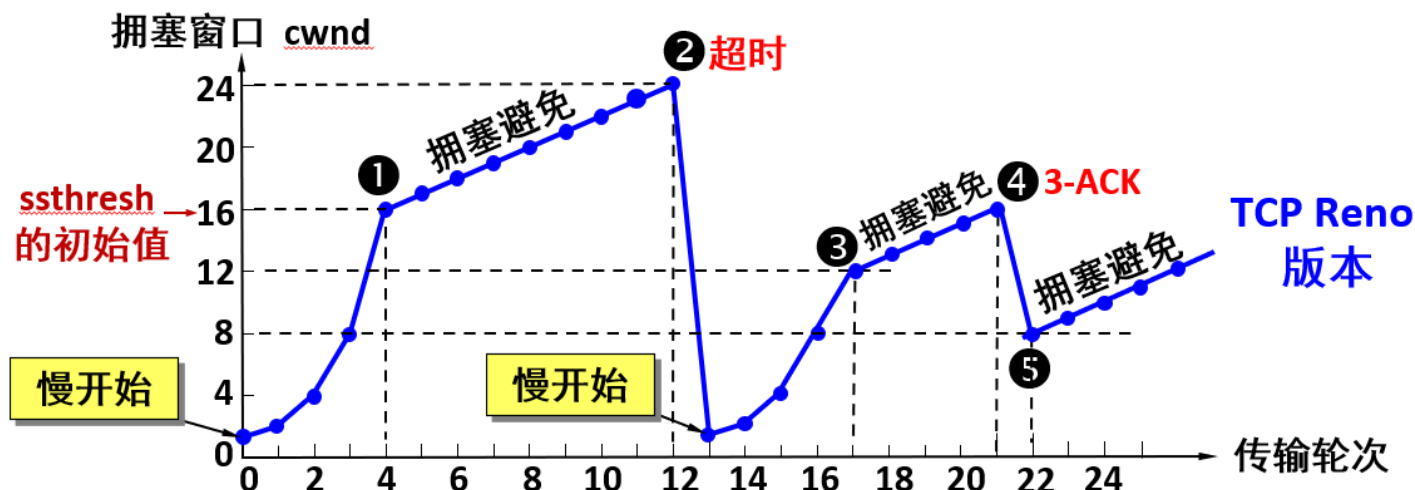
- 已知条件
 - 文件大小: F 比特
 - 对等方数量: N 个
 - 服务器上传速率: u_s 比特/秒
 - 上传速率: u_i 比特/秒
 - 下载速率: d_i 比特/秒
- 客户-服务器体系
 - 服务器上传最短时间: $\frac{NF}{u_s}$
 - 客户下载最短时间: $\frac{F}{d_{min}}$
 - 最小分发时间: $D_{CS} = \max(\frac{NF}{u_s}, \frac{F}{d_{min}})$
- P2P 体系

- 服务器上传最短时间: $\frac{F}{u_s}$
- 对等方下载最短时间: $\frac{F}{d_{min}}$
- 总体上传时间: $\frac{NF}{u_s + \sum u_i}$
- 最小分发时间: $D_{P2P} = \max(\frac{F}{u_s}, \frac{F}{d_{min}}, \frac{NF}{u_s + \sum u_i})$

pipelining



慢开始和拥塞避免



路由器缓存

$$B = \frac{RTT \cdot C}{\sqrt{N}}$$

- 缓存数: B
- 平均往返延时: RTT
- 链路容量: C
- TCP流的数量: N

时隙ALOHA效率

- 节点数: N
- 给定节点在一个时隙中传输的概率: p
- 在一个时隙中, 成功传输的概率: $p(1 - p)^{N-1}$
- N 个节点中任意一个节点成功传输的概率: $Np(1 - p)^{N-1}$
- 最大效率: $G = \lim_{N \rightarrow \infty} Np(1 - p)^{N-1} = \frac{1}{e} \approx 0.37$

"成功传输"意味着一个节点传了, 其他节点没传

术语缩写对照表

中文名	英文名	缩写
名称	name	
主机	host	
端系统	end system	
通信链路	communication link	
分组交换机	packet switch	
分组	packet	
路由器	router	
链路层交换机	link-layer switch	
因特网服务提供商	Internet Service Provider	ISP
传输控制协议	Transmission Control Protocol	TCP
用户数据报协议	User Datagram Protocol	UDP
异步传输模式	Asynchronous Transfer Mode	ATM
网际协议	Internet Protocol	IP
请求评论	Request For Comment	RFC
套接字接口	socket interface	
客户	client	
服务器	server	
往返时间	Round-Trip Time	RTT
转发表	forwarding table	
频分复用	Frequency-Division Multiplexing	FDM
时分复用	Time-Division Multiplexing	TDM
传输层安全	Transport Layer Security	TSL
超文本传输协议	HyperText Transfer Protocol	http
简单邮件传输协议	Simple Mail Transfer Protocol	SMTP
多媒体邮件扩展	Multimedia Mail Extension	MIME
邮局协议第三版	Post Office Protocol Version 3	POP3
域名系统	Domain Name System	DNS
分布式拒绝服务	Distributed Denial of Service	DDOS
基于HTTP的动态自适应流	Dynamic Adaptive Streaming over HTTP	DASH

中文名	英文名	缩写
最大分段大小	Maximum Segment Size	MSS
最大传输单元	Maximum Transmission Unit	MTU
网络地址转换	Network Address Translation	NAT
动态主机配置协议	Dynamic Host Configuration Protocol	DHCP
广域网	Wide Area Network	WAN
局域网	Local Area Network	LAN
深度数据包检查	Deep Packet Inspection	DPI
因特网控制报文协议	Internet Control Message Protocol	ICMP
网络接口卡	Network Interface Card	NIC
地址解析协议	Address Resolution Protocol	ARP
带冲突检测的载波侦听多路访问	Carrier Sense Multiple Access with Collision Detection	CSMA/CD
信噪比	Signal-to-Noise Ratio	SNR
短间隙	Short Interframe Space	SIFS
分布式间隙	Distributed Interframe Space	DIFS
接入点	Access Point	AP